Task-13:-Requirement Analysis

1. Introduction

1.1 Purpose

The purpose of this document is to define the requirements for developing a Linux shell script that automatically deploys an observability stack consisting of **Grafana, Prometheus, Loki, and Mimir** on an AWS EC2 instance.

Currently, installing these monitoring tools manually requires multiple configuration steps including dependency installation, downloading binaries, creating configuration files, and starting services. This manual process can lead to configuration inconsistencies and increased deployment time.

The goal of this project is to automate the deployment process using a **single reusable shell script**, allowing DevOps engineers and system administrators to quickly deploy a basic monitoring environment on a fresh EC2 instance.

Intended Audience

This document is intended for:

- DevOps Engineers
- System Administrators
- Cloud Engineers
- Project Managers
- Technical Reviewers

1.2 Scope

The scope of this project includes designing and implementing a **Linux shell script** that automates the installation and configuration of the following monitoring tools:

- Grafana
- Prometheus
- Loki
- Mimir

The script will perform the following tasks:

- Install required system dependencies
- Download and install the required monitoring tools
- Configure the services using basic configuration files
- Start the services
- Enable services to run automatically on system reboot
- Verify that the services are running correctly

In Scope

- Deployment of Grafana, Prometheus, Loki, and Mimir
- Creation of required directories
- Installation of dependencies
- Basic configuration of services
- Service startup and enablement

Out of Scope

- Advanced dashboard creation in Grafana
- Alert manager configuration
- Multi-node distributed monitoring architecture

- Production-level security hardening
- Integration with external monitoring systems

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
EC2	Elastic Compute Cloud – Virtual server service provided by AWS
Grafana	Visualization platform used for monitoring dashboards
Prometheus	Open-source monitoring system for collecting metrics
Loki	Log aggregation system designed for logs
Mimir	Scalable backend storage for Prometheus metrics
Observability	Ability to monitor system metrics, logs, and performance
Shell Script	A script written in shell language to automate tasks
DevOps	Combination of development and operations practices

1.4 References

The following resources were used while preparing this document:

- Grafana Official Documentation – Technical documentation | Grafana Labs
- Prometheus Official Documentation – Overview | Prometheus
- Loki Official Documentation – Grafana Loki | Grafana Loki documentation
- Grafana Mimir Documentation – Grafana Mimir documentation | Grafana Mimir documentation
- Amazon EC2 Documentation – https://docs.aws.amazon.com/ec2

1.5 Overview

This document provides a detailed analysis of the requirements for automating the deployment of an observability stack on AWS EC2.

The document is structured as follows:

- **Introduction** – Provides the purpose and scope of the project
- **Project Overview** – Describes objectives and background
- **Functional Requirements** – Explains system functionalities and use cases
- **Non-Functional Requirements** – Defines performance, security, and reliability requirements
- **Assumptions and Dependencies** – Lists assumptions and system dependencies
- **Risk Analysis** – Identifies potential risks and mitigation strategies
- **Glossary** – Defines key terms used in the document

---

2. Project Overview

2.1 Project Objectives

The main objectives of this project are:

- Automate the deployment of an observability stack on AWS EC2
- Reduce manual effort in setting up monitoring tools
- Ensure consistent deployment across environments
- Provide quick and reusable monitoring infrastructure
- Enable DevOps teams to monitor system metrics and logs efficiently

---

2.2 Business Goals

The business goals for this project include:

- Improving operational monitoring capabilities
- Reducing deployment time for monitoring environments
- Minimizing configuration errors
- Enhancing infrastructure visibility
- Supporting faster troubleshooting of system issues

2.3 Background

Monitoring and observability are essential components of modern cloud infrastructure. Organizations rely on monitoring tools to track system performance, detect failures, and analyze logs.

ABC Technologies currently requires a monitoring stack consisting of **Grafana, Prometheus, Loki, and Mimir** to visualize metrics and logs from their infrastructure.

However, manual installation of these tools requires significant effort and can lead to inconsistent configurations across environments.

To address this challenge, the organization plans to develop a **Linux shell script that automates the entire deployment process**. This will simplify the setup and ensure that every environment is configured in a consistent and reliable manner.

2.4 Constraints

The following constraints apply to this project:

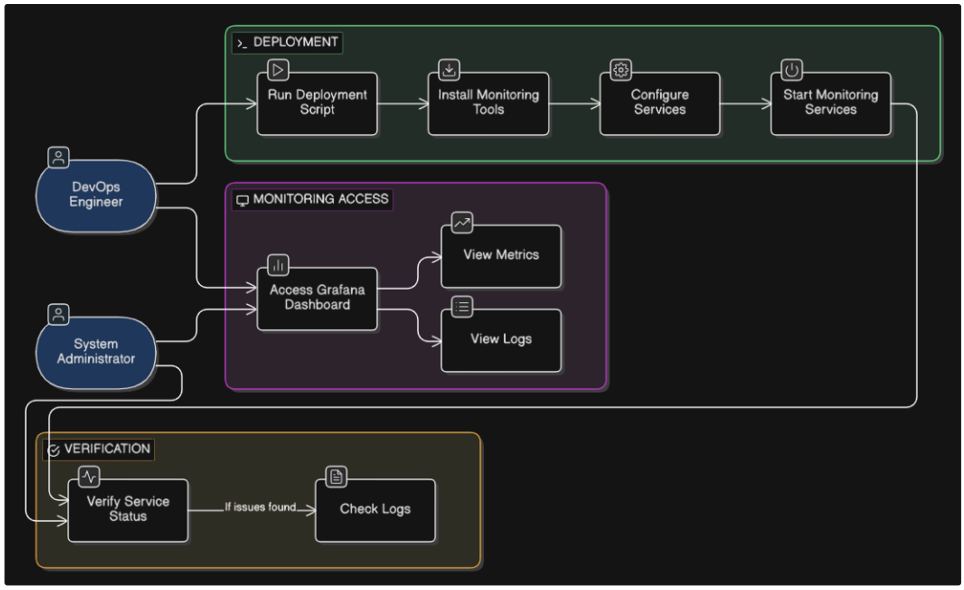
- The deployment must be performed using **Linux shell scripting**
- The monitoring stack must run on a **single EC2 instance**
- The deployment must be completed within **3 hours**
- The configuration should remain **simple and functional**
- The EC2 instance must have internet access for downloading packages

---

3. Functional Requirements

3.1 Use Case Diagram

Insert Use Case Diagram Here



Example actors and actions:

Actor: DevOps Engineer

Use Cases:

- Run Deployment Script
- Install Monitoring Tools
- Configure Services
- Start Monitoring Services
- Access Grafana Dashboard
- Verify Service Status

3.2 Use Cases

3.2.1 Use Case 1: Run Deployment Script

Description

The DevOps engineer executes the deployment script on a fresh EC2 instance to install and configure the observability stack.

Actors

DevOps Engineer

Preconditions

- EC2 instance is running
- User has sudo privileges
- Internet connectivity is available

Postconditions

- All monitoring services are installed
- Services are started
- Services are enabled for automatic startup

Flow of Events

1. User connects to EC2 instance using SSH
2. User downloads the deployment script

3. User executes the script
4. Script installs dependencies
5. Script installs monitoring tools
6. Script configures services
7. Script starts all services

Alternative Flows

Alternative Flow 1:
If internet connectivity is unavailable, package installation fails.

Alternative Flow 2:
If the script lacks execution permission, the user must run:

```
chmod +x deploy.sh
```

3.2.2 Use Case 2: Access Grafana Dashboard

Description

Users access Grafana through a web browser to visualize monitoring data.

Actors

DevOps Engineer
System Administrator

Preconditions

- Grafana service is running
- EC2 security group allows port **3000**

Postconditions

- User successfully logs into Grafana
- Dashboards can be created and viewed

Flow of Events

1. User opens a web browser
2. User enters EC2 public IP with port 3000
3. Grafana login page appears
4. User logs in using default credentials
5. Grafana dashboard interface is displayed

Alternative Flows

If the port is blocked in the security group, the dashboard will not load.

3.2.3 Use Case 3: Verify Monitoring Services

Description

The DevOps engineer verifies that Prometheus, Loki, Mimir, and Grafana services are running.

Actors

DevOps Engineer

Preconditions

- Deployment script execution completed

Postconditions

- All services are confirmed to be running

Flow of Events

1. User runs service status commands
2. System displays service status
3. User verifies that all services are active

Example command:

```
systemctl status grafana-server
systemctl status prometheus
systemctl status loki
systemctl status mimir
```

Alternative Flows

If any service is inactive, the user can restart it using:

```
systemctl restart <service-name>
```

---

4. Non-Functional Requirements

4.1 Performance Requirements

- The deployment script should complete execution within **10–15 minutes**
- Services should start within **5 seconds**
- Grafana dashboard should load within **2 seconds**

4.2 Security Requirements

- Only authorized users should access the EC2 instance
- Security groups must restrict access to necessary ports only
- Default credentials should be changed after installation
- SSH access should be secured using key-based authentication

---

4.3 Usability Requirements

- The deployment script should be easy to execute
- Clear comments should be included in the script
- Configuration files should be organized in separate directories

---

4.4 Reliability Requirements

- Services should automatically restart after system reboot
- The deployment process should handle errors gracefully
- Logs should be available for troubleshooting

4.5 Compliance Requirements

The deployment should comply with:

- AWS infrastructure usage guidelines
- Organizational DevOps standards
- Basic security best practices

---

5. Assumptions and Dependencies

5.1 Assumptions

- EC2 instance is running Ubuntu Linux
- User has administrative privileges
- Internet connectivity is available
- Required ports are open in the security group

5.2 Dependencies

The project depends on:

- AWS EC2 infrastructure
- Internet access for downloading binaries
- Linux package manager
- Monitoring tools repositories

---

6. Risk Analysis

6.1 Risk Identification

Risk	Description
Internet failure	Packages cannot be downloaded
Port restrictions	Monitoring dashboards cannot be accessed
Incorrect configuration	Services may fail to start
Resource limitation	EC2 instance may not handle workload

6.2 Risk Mitigation

Risk	Mitigation Strategy
Internet failure	Verify network connectivity before execution
Port restrictions	Configure EC2 security groups properly
Incorrect configuration	Test configuration files before deployment
Resource limitation	Use appropriate EC2 instance type

---

7. Glossary

Term	Meaning
Observability	Ability to understand system behavior using metrics and logs

Monitoring	Continuous tracking of system performance
Metrics	Quantitative measurements of system performance
Logs	Recorded system events used for debugging

---

8. Appendices

Additional supporting information may include:

- Deployment script
- Configuration files
- Architecture diagrams
- Screenshots of the monitoring dashboards

---

9. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____